

Analysis: Image Labeler

First, we need to understand how to obtain the median of a set. A [median is the value separating the higher and lower halves](#) of a set, so a set of length k will have its median at element $k/2$. For sets with an even number of elements, the median is the average of the two middle elements.

A brute force approach to this problem is described in the example case, where we try each possible combination of assigning regions to categories, and taking the combination with the largest sum of medians.

Time complexity: $O(2^M)$ since we need to test each combination of either assigning a region or not to each category. We can reduce the time complexity even further through dynamic programming to avoid recalculating assigned categories, but there is an even better approach.

A more efficient approach is to maximize the sum by ensuring that the largest regions each form their separate category. One way to do this is with a greedy approach, where we remove the single largest region (say A_k) and assign it to a new category. Then the median of this category is equal to the size of that region. Note that this is the largest median that we can obtain, because all other regions $A_1, \dots, A_N \leq A_k$. If we continue this process for the next $N - 1$ categories, we will have the largest possible medians for all N regions. The largest regions can be identified by sorting A in descending order.

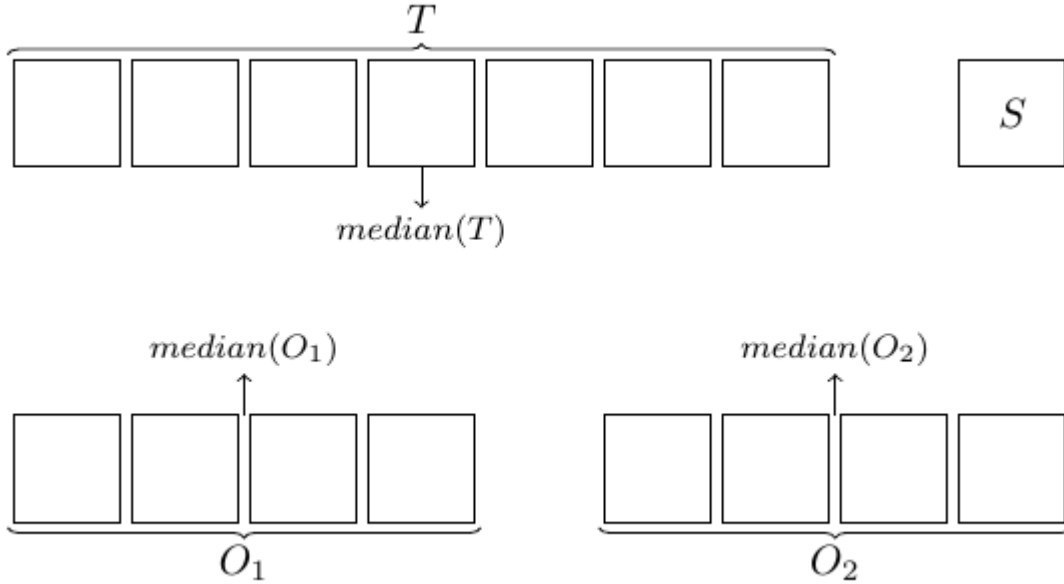
The problem is that we may run out of categories. Precisely, we still have $N - M$ regions that we need to assign to some categories. Since we know that for each of these remaining regions that they are smaller or equal to the ones already assigned to categories, then assigning a new region to an existing category may reduce the median of that category. We can decide to decrease the median of only one category — the one that already has the smallest median. Our result (the sum of medians) is then the sum of the largest $M - 1$ regions plus the median of all other regions.

Time complexity: $O(N \log N + M)$ since we need to sort N regions and sum up the top $M - 1$. Calculating the median of the final category is only $O(1)$ because the regions are already in sorted order. We can even solve this problem in linear time, but it was not required for this problem. We only need to find top $M-1$ regions (and we can do it in $O(N + M)$ time), and then we can find the median of the remaining regions [in linear time](#) as well.

Proof

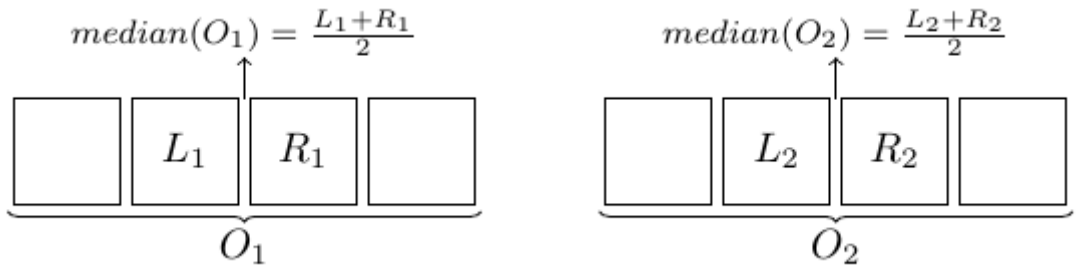
We can formally prove that the greedy algorithm produces an optimal result.

We will prove that given any two arbitrary categories, the sum of medians does not decrease if we reassign the regions within these categories so that the largest region forms one category, and all the other regions forms the other category. Let $\{O_1, O_2\}$ denote these two arbitrary categories. Let S denotes the number of participants in the region with the largest number of participants within these categories and $T = O_1 \cup O_2 \setminus \{S\}$ (S and T are two categories produced by the greedy algorithm). We want to show that $median(O_1) + median(O_2) \leq S + median(T)$. Without loss of generality, let us assume that $median(O_1) \leq median(O_2)$. In the pictures below, let us assume that the regions within a category are sorted in the non-decreasing order.



In T , there are at least $\left\lceil \frac{|O_1|}{2} \right\rceil + \left\lceil \frac{|O_2|}{2} \right\rceil - 1$ regions larger than or equal to $\text{median}(O_1)$ — the regions larger than or equal to the medians of O_1 and O_2 , but we need to remove S (hence -1). Now we have to consider two cases:

1. Either in T there are at least $\left\lceil \frac{|O_1|}{2} \right\rceil + \left\lceil \frac{|O_2|}{2} \right\rceil$ regions larger than or equal to $\text{median}(O_1)$, which implies $\text{median}(T) \geq \text{median}(O_1)$, so we can say that $\text{median}(O_1) + \text{median}(O_2) \leq S + \text{median}(T)$, what we wanted to prove.
2. There are exactly $\left\lceil \frac{|O_1|}{2} \right\rceil + \left\lceil \frac{|O_2|}{2} \right\rceil - 1$ regions larger than or equal to $\text{median}(O_1)$. Now we will once again consider various cases depending on the parity of $|O_1|$ and $|O_2|$:
 1. $|O_1|$ and $|O_2|$ are even. Let us denote L_1 and R_1 as two "middle" regions of O_1 and L_2 and R_2 as two "middle" regions of O_2 .



Then we can use the similar argument as before and say that in T there are at least $\left\lceil \frac{|O_1|}{2} \right\rceil + \left\lceil \frac{|O_2|}{2} \right\rceil$ regions larger than or equal to $\max(L_1, L_2)$, so $\text{median}(T) \geq \min(\max(L_1, L_2), \min(R_1, R_2))$, so $S + \text{median}(T) \geq \frac{L_1 + R_1}{2} + \frac{L_2 + R_2}{2} = \text{median}(O_1) + \text{median}(O_2)$, since $S \geq \max(R_1, R_2)$.

2. $|O_1|$ and $|O_2|$ are odd. Then we can see that there are $\left\lceil \frac{|O_1|}{2} \right\rceil + \left\lceil \frac{|O_2|}{2} \right\rceil$ regions with more participants than $\text{median}(O_1)$, which is the case that we already described above.
3. One of $|O_1|$ and $|O_2|$ is even and the other one is odd. The argument here is similar to the arguments above, so we will leave it as an exercise to a reader.

[equally well for everyone, everywhere](#), you can [download it here](#).

Analysis: Maximum Gain

Test Set 1

One straightforward way to solve this problem is to simulate the process with a recursive brute force method. Let us define the following variables:

- Ai : The pointer to the start of array **A**. The pointer moves one step ahead after answering the first question from **A**, i.e. Ai becomes $Ai + 1$.
- Aj : The pointer to the end of array **A**. The pointer moves one step back after answering the last question from **A**, i.e. Aj becomes $Aj - 1$.
- Bi : The pointer to the start of array **B**.
- Bj : The pointer to the end of array **B**.
- k : The remaining number of questions to answer.

With these variables, we can write down the recursive equation as follows:

$$f(Ai, Aj, Bi, Bj, k) = \begin{cases} -\text{inf}, & \text{if } (Ai - 1) + (N - Aj) > N \text{ or } (Bi - 1) + (M - Bj) > M. \\ 0, & \text{if } k = 0. \\ \max \begin{pmatrix} f(Ai + 1, Aj, Bi, Bj, k - 1) + \mathbf{A}[Ai] \\ f(Ai, Aj - 1, Bi, Bj, k - 1) + \mathbf{A}[Aj] \\ f(Ai, Aj, Bi + 1, Bj, k - 1) + \mathbf{B}[Bi] \\ f(Ai, Aj, Bi, Bj - 1, k - 1) + \mathbf{B}[Bj] \end{pmatrix}, & \text{otherwise.} \end{cases}$$

The maximum points we can get by answering **K** questions from arrays **A** and **B** would be the return value of $f(1, N, 1, M, K)$. Be careful that indexes Ai , Aj , Bi , and Bj may go out of bounds of arrays **A** and **B**. The out-of-bounds values are defined to be zeros in the equations above.

However, the time complexity of the recursion above is $O(4^K)$. To improve the performance, we can use dynamic programming: keep the return values in a 4-dimensional array to avoid repeated computation. Notice that the memoization array does not need to keep parameter k since it can be derived from the other four parameters with

$k = K - (Ai - 1) - (N - Aj) - (Bi - 1) - (M - Bj)$. With memoization, the time complexity of this method is $O(K^4)$, and space complexity is $O(K^4)$.

Test Set 2

Notice that the order of questions to answer does not affect the points we get from them. Therefore, we can always answer questions in the first array **A** first, and then in the second array **B**. This observation allows us to divide the problem into two sub-problems: find the maximum points P_A we can get by answering K_A questions in the first array **A**, and the maximum points P_B by answering K_B questions in the second array **B**. Enumerating all possible combinations of (K_A, K_B) where $K_A + K_B = K$, the total maximum points we can get by answering **K** questions would be the maximum value of all $P_A + P_B$.

Now we want to find a way to get the maximum points P_A by answering K_A questions in the first array **A**. Since the order of questions to answer does not matter, we can answer K_{AStart} questions from the start of the array first, and then K_{AEnd} questions from the end of the array, such that $K_{AStart} + K_{AEnd} = K_A$. The points we get would be:

$$P'_A = (\mathbf{A}_1 + \mathbf{A}_2 + \dots + \mathbf{A}_{K_{AStart}}) + (\mathbf{A}_{N-K_{AEnd}+1} + \mathbf{A}_{N-K_{AEnd}+2} + \dots + \mathbf{A}_N) \\ = \text{Prefix}(\mathbf{A}, K_{AStart}) + \text{Suffix}(\mathbf{A}, K_{AEnd})$$

Therefore, we can find the maximum points P_A by enumerating all combinations of (K_{AStart}, K_{AEnd}) where $K_{AStart} + K_{AEnd} = \min(K_A, N)$, and find the maximum values of all P'_A with the equation aforementioned. With the prebuilt prefix sum and suffix sum arrays, we can get P'_A in $O(1)$ time, and the maximum points P_A in $O(K_A)$ time.

We can apply the same algorithm to find the maximum points P_B by answering K_B questions in the second array $\mathbf{B}$. The time complexity for this method would be:

- $O(N + M)$ to prebuild the prefix and suffix arrays for $\mathbf{A}$ and $\mathbf{B}$.
- $O(K^2)$ to get the total maximum points.
 - $O(1)$ to get the points with a pair (K_{AStart}, K_{AEnd}) or (K_{BStart}, K_{BEnd}) .
 - $O(K)$ to enumerate all (K_{AStart}, K_{AEnd}) pairs and get P_A — maximum points by answering questions from array $\mathbf{A}$. Same for array $\mathbf{B}$.
 - $O(K^2)$ to enumerate all (K_A, K_B) pairs and get the total maximum points.
- Overall: $O(K^2 + N + M)$.

If you would like to explore more tasks like Audio Validation and Image Labeler Tasks on the Crowdsourcing app and directly train Google's artificial intelligence systems to make Google products work [equally well for everyone, everywhere](#), you can [download it here](#).

Analysis: Suspects and Witnesses

Let each suspect be a node in a graph G . If A is a witness to B , then form a directed edge from A to B .

It can be proven that a [strongly connected component \(SCC\)](#) C in G has the property that either:

1. Everyone is innocent.
2. Or everyone is a cookie stealer.

As a proof, if an arbitrary node u is innocent, then, for every edge e that goes from u to v , node v is also innocent. By induction, every node in C is innocent. Similarly, if an arbitrary node u is a cookie stealer, then every node v , where (u, v) is an edge in G , is also a cookie stealer. By induction, every node in C is a cookie stealer.

Given our graph G , we find all the SCC in G and form a new graph G' where:

1. Each SCC in G corresponds to a node in G' .
2. Each node in G' has a value w which corresponds to the size of the SCC.
3. If there exists an edge (u, v) in G and the two nodes do not share a SCC, there is an edge (u', v') in G' where u' is the corresponding node for u and v' is the corresponding node for v .

You can form G' in $O(|V| + |E|)$ using [Tarjan's strongly connected components algorithm](#).

If there are at least $\mathbf{K}$ suspects that can reach node u in G , then u must be innocent.

To prove this, we use contradiction. If u is a cookie stealer, then every node that can reach u is a cookie stealer. Since there are at least $\mathbf{K}$ of them, this implies at least $\mathbf{K} + 1$ cookie stealers which is a contradiction.

Hence, if a node u in G belongs to a SCC (C_1) of size larger than $\mathbf{K}$, that node must be an innocent person. Therefore, we can iterate over all nodes and check this. However, since a node can belong to a different component (C_2) from which C_1 is reachable, then we have to check the reachability of each node to other nodes.

We can go through all the nodes of G' in topologically sorted order. We can go through all the neighbour nodes, i.e., nodes v that have a directed edge from the current node u to v and, since u is reachable to v , we can add u to the set that belongs to v . Whenever a set reaches the size of $\mathbf{K} + 1$, we can stop adding items to it. We will continue this process according to the topologically sorted order. Since we are going through all nodes (topologically), checking every edge at most once at the worst case and performing at most $\mathbf{K} + 1$ insertions into the set, the overall time complexity for the method would be $O((\mathbf{N} + \mathbf{M})\mathbf{K} \log \mathbf{K})$. Once all nodes have been visited, we can go through all the nodes and count the number of nodes that have a reachability set of size is more than $\mathbf{K}$ which should take $O(\mathbf{N})$.

Therefore, the overall time complexity of this solution would be $O((\mathbf{N} + \mathbf{M})\mathbf{K} \log \mathbf{K})$. In terms of space complexity, we would require $O(\mathbf{N})$ for SCC construction and SCC graph construction. The space complexity of finding reachability sets will be $O(\mathbf{N}\mathbf{K})$ since for every node we can have at most $\mathbf{K} + 1$ nodes that we store. Therefore, the overall space complexity should be $O(\mathbf{N}\mathbf{K})$.

Analysis: Touchbar Typing

Test Set 1

When there is no duplicate key in the keyboard, there is only one way to type any given string. It would suffice to simulate typing the given string and count the cost, for test set 1.

We can build a mapping from each character in the keyboard to its position in the keyboard. Let us denote the position of i -th character of $\mathbf{S}$ on the keyboard as p_i .

Let us denote the current position of the finger on the keyboard with y . We should start with the finger on the keyboard on p_1 position.

Then in each step, we need to add $|y - p_i|$ to the answer, and update the current keyboard position to p_i .

The time complexity of creating the mapping from each character to a position on the keyboard is $O(\mathbf{M})$. With help of the mapping previously mentioned, the time complexity of calculating the answer is $O(\mathbf{N})$. The total time complexity of this approach is $O(\mathbf{N} + \mathbf{M})$, which is sufficient for test set 1.

Test Set 2

In test set 2, keys on the keyboard are not unique. At each step of typing, there might be multiple possible positions on the keyboards to move to next, and making optimal choice at every step will lead to the minimum number of moves.

Let us denote the set of positions of i -th character of $\mathbf{S}$ on the keyboard as P_i .

We can consider a dynamic programming approach, where for each position x on the given string, and each position y on the keyboard, we need to find the optimal solution based on possible options for that instance. Then we have the recurrent function F ,

$$F(x, y) = \min(F(x-1, j) + |(y - j)|), j \in P_{x-1}.$$

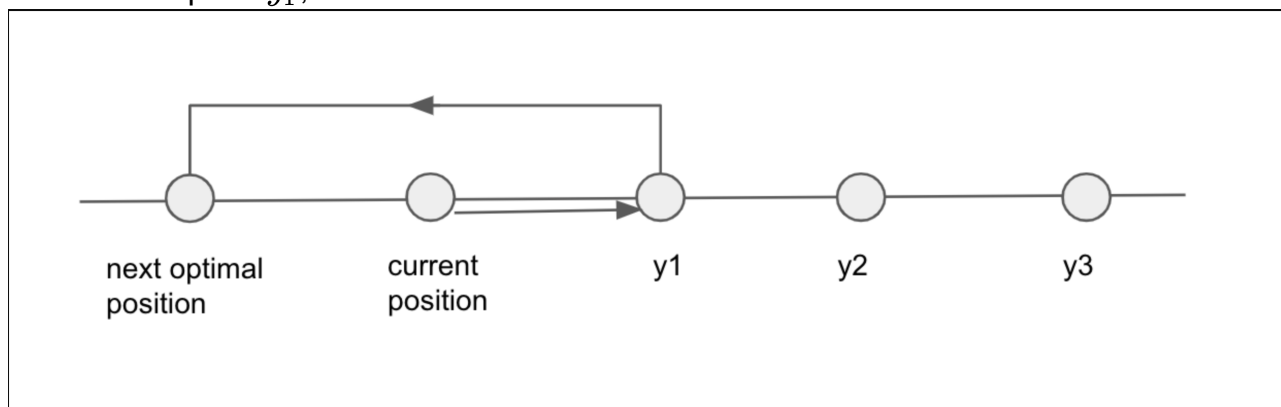
This approach gives us a time complexity of $O(\mathbf{N} \times \mathbf{M} \times \mathbf{M})$, as there are total $\mathbf{N} \times \mathbf{M}$ possible states, and on each state, there are at most $\mathbf{M}$ options in the worst case. This is sufficient for test set 2.

Test Set 3

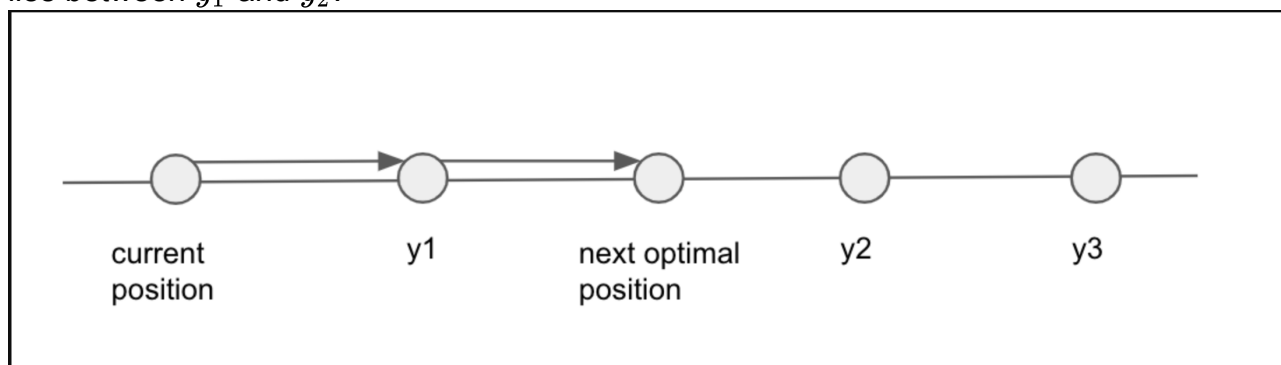
For test set 3, $O(\mathbf{N} \times \mathbf{M} \times \mathbf{M})$ is not sufficient. We need to make the choices more carefully to reduce the complexity. Try to think a little on how to optimise this step before looking into the solution :)

One interesting observation is that every time when there are multiple possible keyboard positions to jump to type the next character, it is always optimal to pick either the one with the minimal distance on the left direction, or the one with the minimal distance on the right direction. Let us consider the current position on the keyboard is x , and there are multiple possibilities on the right, denoted by y_1, y_2, y_3 and so on, sorted by in the order of increasing distance from x . Now, jumping to y_2 or y_3 cannot be a better choice than jumping to y_1 . Let us consider two cases,

1. The next optimal move lies in the opposite direction, aka left. Then it is better to jump to the closest option y_1 , to minimise the total distance.



2. The next optimal move lies in the same direction, aka right. Then jumping to y_1 is at least not worse than jumping to y_2, y_3 . It can even be a better option, if the next optimal position lies between y_1 and y_2 .



The same reasoning is applicable in case of making a move to the left. Then it basically boils down to making a choice of whether to move right, or left on the keyboard from any current position. The recurrent function then changes to F ,

$F(x, y) = \min((F(x - 1, j_{left}) + |y - j_{left}|), (F(x - 1, j_{right}) + |y - j_{right}|))$ where $j_{left}, j_{right} \in P_{x-1}$ and j_{left} and j_{right} are the closest two positions from x on the left and the right respectively.

This makes the number of choices $O(1)$, giving us a total time complexity of $O(\mathbf{N} \times \mathbf{M})$, which is sufficient for test set 3.

If you would like to explore more tasks like Glide Type on the Crowdsourcing app and directly train Google™s artificial intelligence systems to make Google products work [equally well for everyone, everywhere](#), you can [download it here](#).